

Compliance Verification Pipeline

Progress Report – APOLLO Proof Loop & ITA Evaluation

June 15, 2026

Current status. The APOLLO proof loop is complete and has been evaluated on 20 Income Tax Act (ITA) provisions across three ablation configurations. The deterministic tactic ladder closes **89.3% (25/28) of all proof targets**, including **100% of individual provision theorems**. Three composite eligibility gate theorems remain open – the failure mode is understood and the fix is a one-line tactic addition. The same pipeline runs unchanged on the 143-provision SEBI ICDR dataset; that ablation is queued next.

1 The APOLLO Proof Loop

The core of the pipeline is a Python loop (`verify_one_lake.py`) that tries to close each proof target in `TaxProbe.lean` automatically, using Lean’s own compiler as the proof oracle. For each theorem, three phases run in sequence. A phase that succeeds short-circuits the rest.

How the oracle works. The loop patches the `sorry` stub in the probe file with a candidate tactic, runs `lake build TaxProbe`, and reads the exit code. Exit 0 means the kernel accepted the proof; a non-zero exit means the tactic failed. The probe file is always restored to its original `sorry` state before the next attempt, so failures leave no side-effects.

Phase 1 – Deterministic tactic ladder

A fixed ordered list of tactics is tried in sequence. We call this the *ladder*. The basic ladder is:

```
decide, simp[I], simp[I,List.all], omega, norm_num, trivial, rfl
```

where *I* is the concrete instance name (`sample_taxpayer` for ITA, `sample_compliant_issuer` for ICDR). In practice, two patterns close almost everything:

```
-- Nat threshold goals (e.g. days_in_india >= 182)
unfold reg_6_1; dsimp [sample_taxpayer]; omega

-- Bool field goals (e.g. is_stt_paid_on_transfer = true)
unfold reg_111a_stt_check; simp [sample_taxpayer]
```

The extended ladder (Config B) adds `native_decide`, `aesop`, `tauto`, and compound variants like `simp[I]; omega`.

Tactic reference

Table 1 explains each tactic used in the ladder and decompose phases. In all entries, *goal* refers to the current proof obligation that Lean is trying to close.

Table 1: Lean 4 tactics used in the APOLLO proof loop.

Tactic	What it does	When we use it
<code>decide</code>	Evaluates the goal computationally by reducing it to <code>True</code> or <code>False</code> via Lean’s kernel.	Bool field checks on small concrete values; e.g., <code>sample_taxpayer.is_resident = true</code> .
<code>native_decide</code>	Same as <code>decide</code> , but compiles the decision procedure to native code before running it. Much faster on larger computations.	Decidable goals that time out under plain <code>decide</code> .
<code>simp [I]</code>	Applies a rewrite ruleset, including unfolding the instance <code>I</code> , to simplify the goal and then tries to close it.	Bool field goals after struct unfolding; e.g., <code>simp [sample_taxpayer]</code> reduces <code>is_stt_paid_on_transfer</code> to <code>true</code> and closes.
<code>dsimp [I]</code>	Definitional simplification: reduces the goal using definitional equalities only, without applying rewrite lemmas. Lighter and faster than <code>simp</code> .	Unfolding the concrete instance before an <code>omega</code> call; e.g., <code>dsimp [sample_taxpayer]</code> replaces <code>days_in_india.fy</code> with <code>365</code> .
<code>omega</code>	Decision procedure for linear arithmetic over integers and naturals. Closes goals that are linear inequalities or equalities.	Nat threshold goals after unfolding; e.g., closes <code>365 >= 182</code> or <code>150000 <= 150000</code> .
<code>norm_num</code>	Normalises numerical expressions. Handles arithmetic that <code>omega</code> may not directly close, such as products or modular arithmetic.	Rate and percentage checks involving products; e.g., <code>125 = 125</code> or <code>10 * 15000 = 150000</code> .
<code>rfl</code>	Closes goals of the form <code>a = a</code> when both sides reduce to the same term by computation.	Literal equality goals where no arithmetic or rewriting is needed; e.g., <code>ltcg_exempt_limit = 125000</code> when the field is directly set to <code>125000</code> .
<code>trivial</code>	Closes goals that are immediately true, such as <code>True</code> , goals provable by <code>rfl</code> , or goals directly dischargeable by the kernel.	Composite gate theorems whose sub-clauses are all Bool checks; the kernel evaluates the conjunction to <code>True</code> and <code>trivial</code> closes it.
<code>tauto</code>	Closes propositional tautologies: goals true by propositional logic alone, without arithmetic or rewriting.	Conjunction and disjunction goals that follow purely from hypothesis structure.
<code>aesop</code>	General-purpose proof search tactic that combines many strategies such as simplification, introductions, case splits, and applications. Slower but more powerful than any single tactic.	Last resort in the extended ladder when simpler tactics fail.
<code>constructor</code>	Splits a conjunction goal <code>A && B</code> or <code>A ∧ B</code> into independent sub-goals, which are then closed separately.	Phase 2 decomposition of composite eligibility gates; <code>constructor <;> simp [I]</code> splits the goal and tries to close each branch in one pass.

Phase 2 – Rule decomposition (APOLLO-style)

Composite eligibility gates conjoin multiple sub-clause predicates:

```
reg_112a_eligible(t) = reg_112a_1(t) ∧ reg_112a_holding(t) ∧ reg_112a_stt_acquisition(t)
```

The ladder cannot close these directly, so Phase 2 tries conjunction-splitting tactics: `constructor` <> `simp [I]`, `unfold-then-simp` variants, and combinations with `omega`. This is the decompose–repair–recombine pattern from APOLLO: split the conjunction into independent sub-goals, close each with the ladder, and let Lean recombine.

Phase 3 – LLM tactic synthesis

If both phases fail, the loop calls Qwen2.5-14B via Ollama with a prompt containing: the current proof goal hint, the provision’s original regulatory text, and field definitions from the extraction step. The suggested tactic is tested via `lake build`. Up to 3 rounds are attempted before the theorem is recorded as unproved.

Three ablation configurations

Config A – No LLM: Basic ladder only. No model calls.

Config B – Extended: Extended ladder. No model calls.

Config C – LLM-assisted: Extended ladder + Qwen2.5-14B fallback.

2 What We Tested

ITA provision corpus

We formalised 20 provisions from the Income Tax Act 1961 covering 10 sections, yielding **28 proof targets** (20 individual provision theorems + 8 composite eligibility gate theorems). All rates and thresholds reflect Finance Act 2024 amendments.

Section	Topic	Predicate type
§6(1)	Residential status (≥ 182 days in India)	$\text{Nat} \geq \text{threshold}$
§80C	Investment deduction cap (₹1.5L)	$\text{Nat} \leq \text{threshold}$
§80D	Health insurance premium (₹25K/50K)	$\text{Nat} \leq + \text{Bool}$
§87A	Tax rebate (income $\leq$ ₹5L/₹7L)	$\text{Nat} \leq \text{threshold}$
§111A	STCG on listed equity (rate 20%, STT paid)	$\text{Bool} + \text{Nat}$
§112A	LTCG on listed equity (exemption ₹1.25L)	Nat arithmetic
§139(1)	Return filing obligation	$\text{Nat} > \text{threshold}$
§194	TDS on dividends (10%, or 20% without PAN)	$\text{Nat} + \text{Bool}$
§194A	TDS on interest (bank threshold ₹40K/50K)	$\text{Nat} + \text{Bool}$
§234B	Advance tax interest (90% of assessed tax)	Nat arithmetic

Sample taxpayer instance

The concrete instance is a resident individual (old tax regime): STCG ₹50,000 on listed equity (STT paid); LTCG ₹2,00,000 on listed equity held 18 months (STT paid at acquisition); dividend income ₹15,000; bank interest ₹33,000; total income ₹12,00,000; §80C investment ₹1,50,000; health insurance premium ₹20,000; advance tax paid ₹45,000; assessed tax ₹82,500. All values are extracted from a synthetic `taxpayer_fields.json` that mirrors a real ITR-2 XML filing.

3 Results

Ablation summary

Table 2: Ablation results on 28 ITA proof targets.

Config	Proved	%	Sub-clauses	Composites	LLM calls	Avg time (s)
A: No LLM	25/28	89.3	20/20	5/8	0	8.93
B: Extended	25/28	89.3	20/20	5/8	0	9.30
C: LLM-assisted	25/28	89.3	20/20	5/8	3	9.08

All three configs produce the same result. The ladder drives everything – neither the extended tactic set (Config B) nor Qwen (Config C) adds proofs over the basic ladder.

Tactic patterns

Every proved theorem falls into one of three patterns:

Table 3: Tactic patterns for the 25 proved theorems.

Goal type	Winning tactic	Count
Nat threshold	<code>unfold {def}; dsimp [sample_taxpayer]; omega</code>	10
Bool field	<code>unfold {def}; simp [sample_taxpayer]</code>	10
Composite gate	<code>trivial</code>	5
Total		25

The split is exact: 10 Nat-threshold provisions (residential days, tax rates, deduction caps, advance tax ratios) all close with the arithmetic pattern; 10 Bool-field provisions (STT flags, PAN status, residency flag, regime choice) all close with `simp` rewriting; and 5 composite gates whose sub-clauses are purely Bool reduce to `True` and close with `trivial`.

The three failing theorems

Table 4: Unproved composite gate theorems.

Theorem	Section	Conjunction structure
<code>gold_reg_6_eligible</code>	§6 residential status	<code>Nat ∧ Bool</code>
<code>gold_reg_112a_eligible</code>	§112A LTCG eligibility	<code>Nat ∧ Nat ∧ Bool</code>
<code>gold_reg_80d_eligible</code>	§80D health insurance	<code>Nat ∧ Bool</code>

These three gates conjoin at least one Nat comparison sub-clause with at least one Bool sub-clause. The Nat branches individually close with `dsimp [...] ; omega`, but inside a `constructor` split, `dsimp` is not being applied before `omega`. The compound tactic that would close them is:

```
constructor <> (unfold {sub_clauses}; dsimp [sample_taxpayer]; try
  omega;
  try simp [sample_taxpayer])
```

This is not currently in the Phase 2 decompose tactic list. Adding it is the immediate fix (see Open Items).

The Qwen fallback (Phase 3) was invoked once per failing gate (3 calls total) and proposed tactics that did not close any goal. This is expected: the prompt gives Qwen the high-level goal `goal ⊢ reg_6_eligible sample_taxpayer` but not the post-split sub-goals. Without seeing the Nat sub-goal, the model cannot suggest `dsimp; omega`. This is a prompt-engineering gap, not a model capability gap.

Timing

Average time is ≈ 9 seconds per theorem across all configs. Sub-clause theorems close in 1.4–2.9 seconds (one or two `lake build` calls). Composite gates that succeed with `trivial` take 17–21 seconds because `trivial` sits 7th in the ladder and six prior build calls fail first. The three failing gates take 34–40 seconds as the oracle exhausts the full tactic sequence. Config B is marginally slower (9.30s vs. 8.93s) because the extended ladder has more tactics to try.

4 Key Findings

1. The deterministic ladder is the workhorse. A two-pattern ladder – arithmetic unfold for Nat goals, `simp` rewrite for Bool goals – closes all 20 individual provision theorems with zero model calls. This holds because each individual regulatory provision tests one or two concrete typed fields against a threshold or flag, and Lean’s kernel decides such goals by reduction.

2. 100% on sub-clauses, 62.5% on composite gates. The split is structurally informative. Individual provisions are essentially *decidable* once unfolded; the kernel’s reduction engine handles them completely. Composite gates are conjunctions of individual results, and the current Phase 2 decompose tactics are not yet complete for mixed-type conjunctions.

3. LLM contribution is zero at current prompt granularity. Config C adds no proofs over Config A. This is not a model failure – it is a diagnostic: the residual failures need a structural tactic (`dsimp` inside a constructor branch), not domain knowledge. Qwen cannot suggest `dsimp` without seeing the post-`constructor` proof state. Integrating LeanDojo’s Dojo interface (which exposes live proof states) would unblock this.

4. Cross-domain: one pipeline, two regulatory frameworks. The ICDR library (`Reglib_gold`) was built with the same pipeline by changing three CLI flags. Lean built cleanly on 143 provisions. The ICDR ablation is queued but not yet run.

5. End-to-end automation gap. ITA is fully automated (structured XML filing $\rightarrow$ fields $\rightarrow$ proof). ICDR still uses manually curated gold-standard rules because IPO prospectuses are unstructured PDFs. Automated ICDR extraction is the main remaining engineering gap for a deployable system.

Open Items

1. **Fix 3 failing composite gates.** Add to `verify_one_lake.py` Phase 2 decompose list:

```
"constructor <;> (unfold {subs}); dsimp [sample_taxpayer]; "  
"try omega; try simp [sample_taxpayer])"
```

Expected: closes `gold_reg_6_eligible`, `gold_reg_112a_eligible`, and `gold_reg_80d_eligible` → 28/28 (100%) on ITA.

2. **Run ICDR ablation.** Generate `GoldProbe.lean` in `Reglib_gold`, commit, and run all three configs. Compare ICDR vs. ITA success rates – ICDR is predominantly Bool fields, so expect higher ladder-only rate.
3. **Integrate LeanDojo Dojo for live proof states.** Run Branch B (pre-populated WSL cache) and wire the post-constructor proof state into the Qwen prompt. This should convert the 3 structural failures into LLM-assisted proofs and validate Phase 3 properly.
4. **Build automated ICDR extraction.** Design an LLM extractor from RHP PDFs that outputs the same JSONL format as the gold standard. This closes the end-to-end pipeline gap for ICDR.
5. **Expand ITA corpus.** Add DTAA provisions, carry-forward loss rules, and alternative minimum tax. These have more complex proof structure and will stress-test the pipeline beyond threshold checks.

A Pipeline Diagram

Figure 1 shows the full two-domain pipeline for reference.

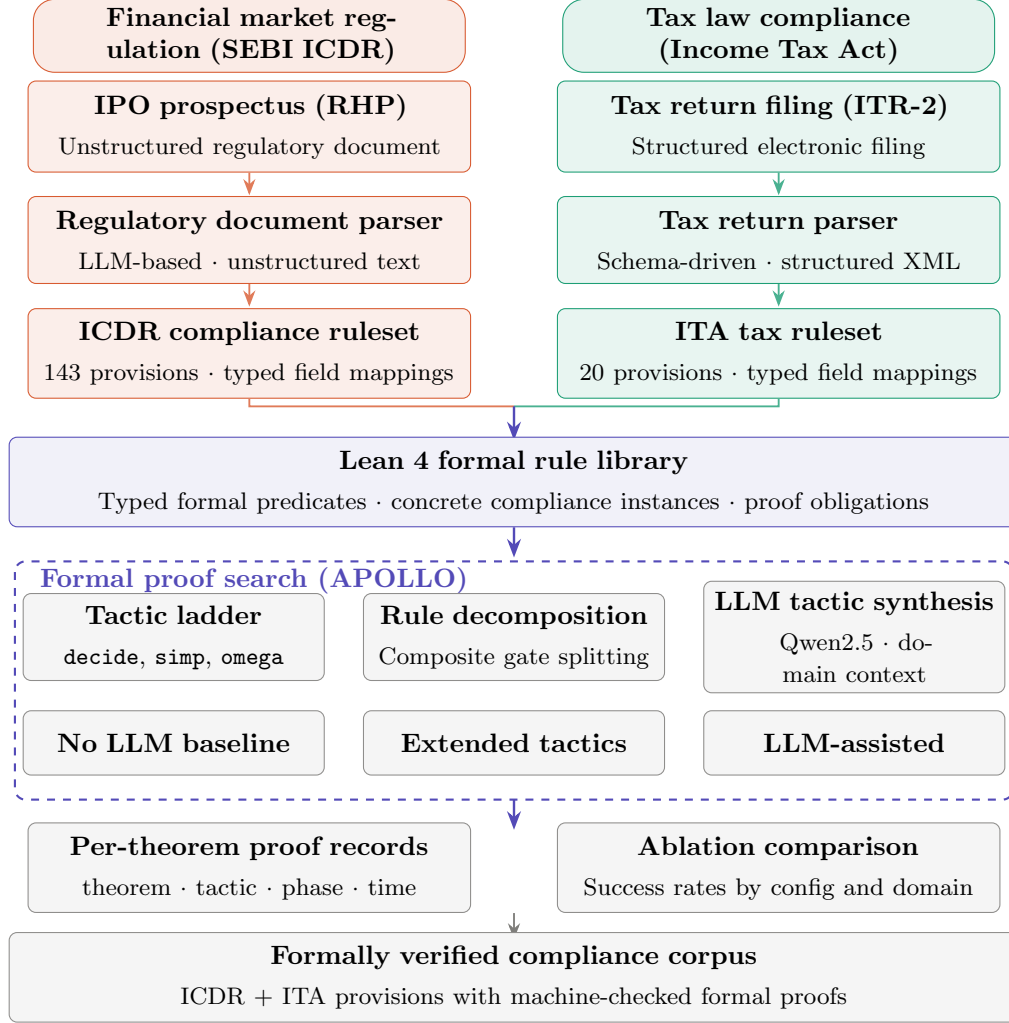


Figure 1: Full two-domain pipeline (system view). The two lanes converge at the Lean 4 library generation step. All proof search logic is shared.

Figure 2 summarizes the end-to-end formal compliance verification architecture.

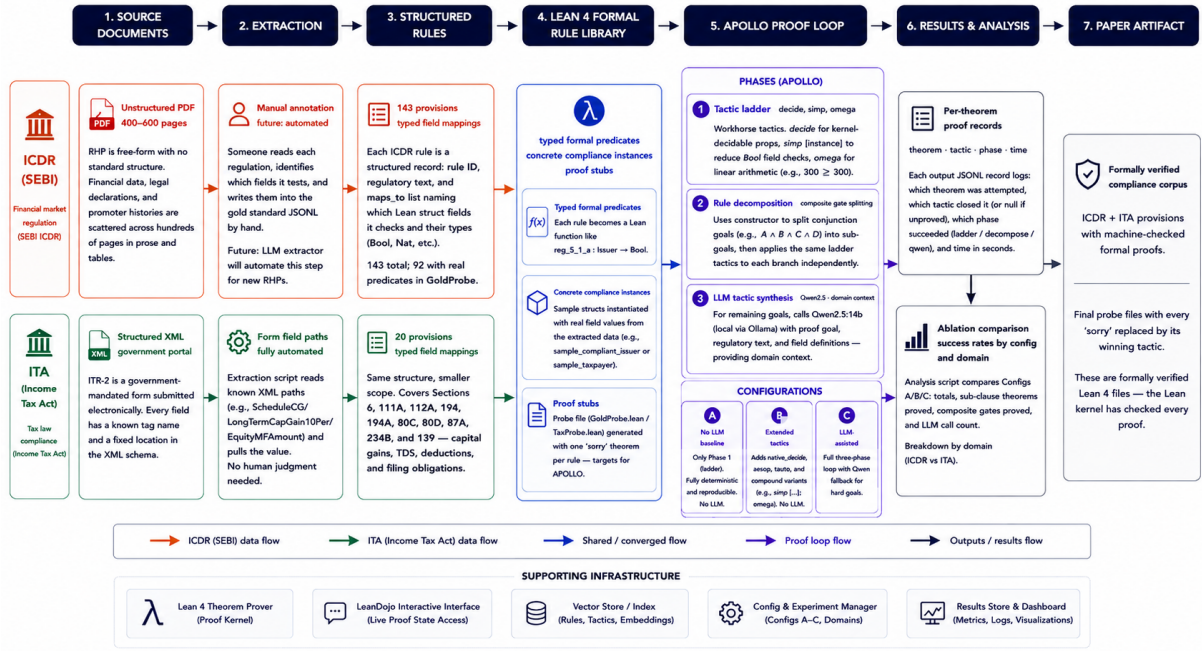


Figure 2: System architecture for the formal compliance verification pipeline.